

GIU Berlin: IET Networks

Bachelor Thesis

# **MANTA: Can We Detect Threats Without Seeing the Payload?**

A public-corpus study of metadata-only anomaly detection, release  
privacy, and observer leakage

|                   |                      |
|-------------------|----------------------|
| Author:           | Mahmoud Elfeel       |
| Supervisor:       | Prof. Mohamed Ashour |
| Industry Partner: | Resistine GmbH       |
| Submission Date:  | July 2026            |

# Abstract

This thesis investigates MANTA (Mobile Anomaly and Network Threat Analysis), an anomaly-first hybrid intrusion-detection system for encrypted mobile and endpoint traffic that operates on flow metadata rather than packet payloads. The implemented artifact spans three modules: an Android endpoint based on **VpnService** [5], a backend adapter for authenticated ingest and analyst feedback, and a reproducible machine-learning pipeline for training, privacy benchmarking, and model export.

The evaluation uses a public-only merged corpus of 6,034,487 flows drawn from seven dataset sources and 904,371 aggregated 60-second windows. It compares on-device linear and TFLite-compatible detectors, remote hybrid models, privacy-student variants, federated simulation, and privacy-reduced feature views under grouped, source-aware splits. The strongest anomaly result in the March 2026 matrix is a remote gradient-boosted tree with F1 0.449, PR-AUC 0.482, and ROC-AUC 0.765, while lighter on-device models remain substantially weaker.

The privacy evaluation separates two attack surfaces. *Release leakage* is inference from the feature representation that the system intentionally exports or shares. *Observer leakage* is inference from the encrypted traffic sequence visible to an on-path network observer before any export-time reduction is applied. The reduced privacy views (**medium** and **strict**) preserve anomaly utility in the March 2026 ablation study and lower exact app-identification leakage from 0.235 normalized leakage in the full view to 0.113 and 0.099, respectively. However, stronger metadata leakage remains high for app family, destination behavior, and dataset source, and the encrypted-flow sequence audit shows severe observer leakage, with normalized leakage 0.544 for app identification and 0.981 for dataset-source inference. I conclude that feature-level coarsening and privacy-aware representation learning improve the released representation, but do not hide the traffic patterns available to passive observers.

# Contents

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>Abstract</b>                                                       | <b>i</b> |
| <b>1 Introduction</b>                                                 | <b>1</b> |
| 1.1 Motivation . . . . .                                              | 1        |
| 1.2 Problem Statement . . . . .                                       | 1        |
| 1.3 Research Questions . . . . .                                      | 2        |
| 1.4 Contributions . . . . .                                           | 2        |
| 1.5 Thesis Structure . . . . .                                        | 3        |
| <b>2 Background and Related Work</b>                                  | <b>4</b> |
| 2.1 Android Networking and VpnService . . . . .                       | 4        |
| 2.2 Flow-Based Monitoring and Metadata-Only Detection . . . . .       | 4        |
| 2.3 Encrypted Traffic Analysis and Observer Leakage . . . . .         | 5        |
| 2.4 Anomaly Detection for Network Security . . . . .                  | 5        |
| 2.5 Privacy-Preserving Traffic Representation . . . . .               | 6        |
| 2.6 Privacy-Aware Representation Learning and Collaboration . . . . . | 6        |
| 2.7 Positioning . . . . .                                             | 6        |
| <b>3 System Architecture and Design</b>                               | <b>7</b> |
| 3.1 System Overview . . . . .                                         | 7        |
| 3.2 Data Flow . . . . .                                               | 8        |
| 3.3 Threat Model and Assumptions . . . . .                            | 8        |
| 3.4 Privacy Model . . . . .                                           | 9        |
| 3.5 Privacy-Preserving Design Decisions . . . . .                     | 9        |
| 3.6 Deployment and Evaluation Boundaries . . . . .                    | 10       |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>4</b> | <b>Implementation</b>                                      | <b>11</b> |
| 4.1      | Android Endpoint Implementation . . . . .                  | 11        |
| 4.1.1    | VPN Capture and Userspace Forwarding . . . . .             | 11        |
| 4.1.2    | Feature Windows and On-Device Detection . . . . .          | 11        |
| 4.1.3    | UI and Triage Workflow . . . . .                           | 12        |
| 4.2      | Backend Adapter Implementation . . . . .                   | 12        |
| 4.3      | ML Pipeline Implementation . . . . .                       | 12        |
| 4.3.1    | Privacy Views . . . . .                                    | 13        |
| 4.3.2    | Privacy Student and Federated Path . . . . .               | 13        |
| 4.3.3    | Reproducibility Support . . . . .                          | 13        |
| <b>5</b> | <b>Experimental Methodology and Evaluation</b>             | <b>14</b> |
| 5.1      | Experimental Setup . . . . .                               | 14        |
| 5.2      | Datasets and Trace Collection . . . . .                    | 14        |
| 5.3      | Metrics . . . . .                                          | 15        |
| 5.4      | Anomaly-Detection Results . . . . .                        | 15        |
| 5.5      | Release-Privacy Utility Trade-off . . . . .                | 16        |
| 5.6      | Metadata Leakage from Released Views . . . . .             | 16        |
| 5.7      | Observer Leakage from Encrypted Traffic . . . . .          | 17        |
| 5.8      | Privacy Student, Federated Path, and Performance . . . . . | 17        |
| 5.9      | Interpretation . . . . .                                   | 18        |
| <b>6</b> | <b>Privacy, Ethics, and Limitations</b>                    | <b>19</b> |
| 6.1      | Privacy Impact . . . . .                                   | 19        |
| 6.2      | Ethical Considerations . . . . .                           | 19        |
| 6.3      | Release Privacy versus Observer Privacy . . . . .          | 20        |
| 6.4      | Interpretation of the Negative Result . . . . .            | 20        |
| 6.5      | Leakage and Utility Evaluation . . . . .                   | 20        |
| 6.6      | Limitations and Threats to Validity . . . . .              | 21        |
| <b>7</b> | <b>Conclusion and Future Work</b>                          | <b>22</b> |
| 7.1      | Conclusion . . . . .                                       | 22        |
| 7.2      | Future Work . . . . .                                      | 23        |

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>A</b> | <b>Reproducibility and Artifact Package</b> | <b>24</b> |
| A.1      | Artifact Package . . . . .                  | 24        |
| A.2      | Core Reproduction Commands . . . . .        | 25        |

# List of Tables

|     |                                                                          |    |
|-----|--------------------------------------------------------------------------|----|
| 5.1 | Public-only corpus composition. . . . .                                  | 15 |
| 5.2 | Representative model results from the March 2026 report set. . . . .     | 16 |
| 5.3 | Anomaly utility under privacy-reduced views. . . . .                     | 16 |
| 5.4 | Strongest metadata leakage by privacy view (normalized leakage). . . . . | 17 |
| 5.5 | Observer leakage from encrypted-flow sequence fingerprinting. . . . .    | 17 |
| 5.6 | Selected thesis gates and measured outcomes. . . . .                     | 18 |

# List of Figures

|     |                                                                           |   |
|-----|---------------------------------------------------------------------------|---|
| 3.1 | High-level MANTA artifact path from endpoint capture to thesis evidence.  | 7 |
| 3.2 | Difference between release privacy and observer privacy in MANTA. . . . . | 9 |

# Chapter 1

## Introduction

### 1.1 Motivation

Mobile endpoints continuously communicate across Wi-Fi, cellular, enterprise, and home networks, yet defenders rarely have visibility into those flows without either rooting the device or inspecting sensitive payload content. Android’s `VpnService` provides a practical middle ground by enabling user-space packet interception on non-rooted devices [5]. This makes metadata-only monitoring possible, but it also creates two competing requirements: detection quality must be operationally useful, and data collection must stay meaningfully privacy-constrained.

This tension is especially acute for encrypted traffic. Payload encryption removes content but not all side channels. Flow timing, volume, burst shape, destination characteristics, and session structure can still reveal application identity or user behavior. I therefore treat privacy as two separate questions: what can be inferred from the representation MANTA releases, and what can be inferred by a passive observer that sees the encrypted traffic on the wire.

### 1.2 Problem Statement

I study whether a practical, metadata-only IDS can be built and evaluated as an end-to-end artifact while keeping the privacy boundary explicit. The project asks whether MANTA can:

1. capture and aggregate application-attributed flow metadata without payload inspection,
2. support anomaly scoring across on-device, remote, privacy-student, and federated variants,
3. quantify the privacy properties of released traffic representations, and

4. distinguish those release-privacy properties from observer leakage visible in encrypted traffic sequences.

The evaluation follows the same artifact through detection, release-view reduction, leakage attacks, observer fingerprinting, and reproducibility packaging. That gives one place to compare utility, privacy, and operating constraints instead of treating them as separate experiments.

## 1.3 Research Questions

- RQ1: To what extent can metadata-only flow windows support anomaly detection across a heterogeneous public encrypted-traffic corpus under grouped, source-aware evaluation?
- RQ2: How do on-device, remote, privacy-student, and federated variants compare in terms of anomaly-detection quality and practical deployment readiness?
- RQ3: What privacy-preserving properties do the reduced privacy views actually provide for *released* traffic representations?
- RQ4: How different are those release-privacy properties from privacy against a passive observer of encrypted traffic sequences?

## 1.4 Contributions

- An end-to-end implementation that connects Android metadata capture, backend triage, and a reproducible machine-learning pipeline.
- A public-only training and evaluation corpus assembled from Android malware, enterprise, IoT, campus, and mobile-app traffic datasets with source/session-aware metadata carried through the entire pipeline.
- An evaluation protocol that compares on-device, remote, privacy-student, and federated variants under grouped source-aware holdouts instead of optimistic random splits.
- A split privacy methodology that distinguishes release privacy from observer leakage and measures both explicitly through metadata leakage attacks and encrypted-flow sequence audits.
- An empirical result showing that metadata coarsening can reduce exact app-ID leakage while still leaving substantial app-family, destination-behavior, and observer leakage.

## 1.5 Thesis Structure

Chapter 2 summarizes Android networking constraints, encrypted-traffic analysis, privacy fingerprinting, and privacy-aware learning literature. Chapter 3 defines the system architecture, trust boundaries, and the distinction between release privacy and observer privacy. Chapter 4 details the implementation of the Android endpoint, backend adapter, privacy views, privacy student, and federated path. Chapter 5 presents the public-corpus evaluation methodology and results. Chapter 6 discusses privacy-preserving properties, ethics, and limitations. Chapter 7 closes with future work.

# Chapter 2

## Background and Related Work

### 2.1 Android Networking and VpnService

Android exposes packet interception on non-rooted devices through `VpnService`, which creates a user-space TUN interface and routes device traffic through an application-controlled forwarding path [5]. This design enables endpoint telemetry without kernel modification, but imposes practical constraints:

- a foreground service is required for long-running capture reliability [3],
- upstream sockets must be protected from tunnel recursion (`protect(...)`),
- packet processing must remain resource-aware to avoid user-visible overhead.

For this reason, MANTA restricts collection to metadata-level features and performs lightweight aggregation on-device rather than deep packet inspection.

### 2.2 Flow-Based Monitoring and Metadata-Only Detection

Flow telemetry abstracts packets into structured records keyed by protocol/endpoint tuples and timing/counter statistics. NetFlow v9 and IPFIX formalize this approach and define reusable fields for interoperable flow exchange [8–10, 14]. Compared to payload inspection, flow representations:

- reduce sensitive content exposure,
- compress observations for efficient local storage and export,

- retain behaviorally meaningful signals (volume asymmetry, burstiness, novelty, and frequency change).

Recent anomaly-detection systems such as Kitsune and N-BaIoT show that compact, flow-like representations can still be effective for intrusion detection when combined with lightweight autoencoder or ensemble machinery [18, 19]. MANTA adopts the same metadata-only philosophy, but targets encrypted mobile and endpoint traffic while emphasizing privacy-sensitive deployment constraints.

## 2.3 Encrypted Traffic Analysis and Observer Leakage

Encrypted transport does not eliminate traffic analysis. Website-fingerprinting work has repeatedly shown that timing, size, and direction patterns can reveal a user’s activity even when payloads are fully encrypted. Deep Fingerprinting demonstrated that deep-learning attackers can break many lightweight defenses [15], and subsequent systems research has shown that efficient padding schemes remain difficult to deploy robustly in realistic settings [17]. Similar issues appear in mobile traffic: partial encrypted traces can support user-activity detection [21], and concurrent-flow relationships can be used to classify mobile applications from encrypted traffic [26].

These attacks motivate the release/observer split used in MANTA. A coarsened or anonymized export can reduce what a data consumer learns, while the original encrypted sequence remains visible to an on-path observer. I evaluate those two surfaces separately.

## 2.4 Anomaly Detection for Network Security

Anomaly detection commonly operates in limited-label settings, where unsupervised baselines are attractive because they can be trained on predominantly normal traffic [7]. Isolation Forest remains a practical reference method for tabular anomaly scoring due to strong performance/complexity trade-offs [16]. In operational security settings, threshold policy is often more consequential than raw score quality because base rates and alert costs strongly affect utility [22].

On-device deployment introduces additional constraints: inference must be lightweight, robust under battery pressure, and explainable enough for human triage. MANTA therefore combines statistical, linear, and TFLite-compatible scoring with lightweight feature-contribution summaries that can be generated during inference.

## 2.5 Privacy-Preserving Traffic Representation

Traditional anonymization work such as prefix-preserving pseudonymization protects explicit identifiers but may still leak semantics through traffic structure. PD-PAn argues that preserving utility while defending against semantic attacks requires attention to both prefix relationships and distributional characteristics [13]. MANTA follows that broader view: it does not only hash identifiers, but also evaluates what remains inferable from reduced feature views after identifiers and strong destination hints have been coarsened or removed.

## 2.6 Privacy-Aware Representation Learning and Collaboration

Self-supervised and reconstruction-based learning for encrypted traffic have recently become promising tools for anomaly detection under limited labels [25]. At the same time, federated-learning research has explored how intrusion-detection models can be improved without centralizing raw data [1, 2]. MANTA uses these ideas in a limited but concrete form: a privacy student is trained from fuller supervision onto a reduced view, and a federated simulation operates on that learned latent space rather than on raw full-feature tables.

## 2.7 Positioning

MANTA differs from earlier work in three ways. First, it integrates Android capture, backend triage, privacy views, privacy attack benchmarks, and manuscript-ready evidence generation into one artifact set. Second, it evaluates a public-only corpus with grouped source-aware splits rather than relying on a small controlled trace or a purely random split. Third, it treats privacy as two separate empirical questions: what the released representation leaks, and what a passive encrypted traffic observer can still infer.

# Chapter 3

## System Architecture and Design

### 3.1 System Overview

The system consists of three modules:

- Android endpoint app (**VpnService**, flow capture, on-device scoring, UI).
- Backend adapter (authenticated ingest, queue/retry, triage, policy APIs, optional SIEM forwarding).
- ML pipeline (feature engineering, training, evaluation, threshold calibration, model export).

The architectural goal is to maximize local autonomy while still supporting optional centralized analysis and reproducible experimentation. Detection must remain possible without backend connectivity, but the same feature and policy definitions must also support export, privacy-view derivation, leakage benchmarking, and model retraining.

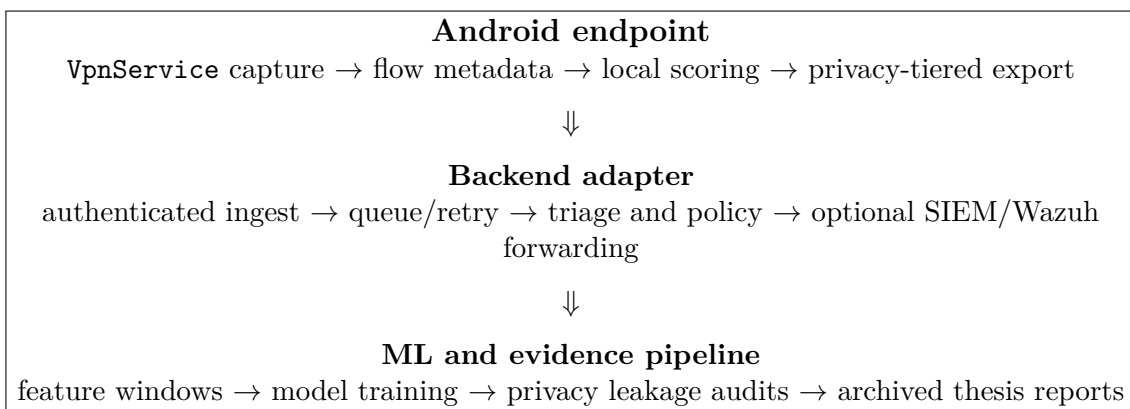


Figure 3.1: High-level MANTA artifact path from endpoint capture to thesis evidence.

## 3.2 Data Flow

The runtime data path is:

1. Packet capture via `VpnService` TUN interface.
2. Userspace forwarding to preserve normal network connectivity.
3. Packet parsing and app attribution to flow records.
4. Per-app rolling window aggregation (60 s default).
5. Local anomaly scoring (ensemble/statistical/linear/TFLite mode).
6. Severity resolution + stability guards + false-positive budget checks.
7. Local persistence of flows/windows/alerts in Room.
8. Optional export of selected flow and alert events to backend APIs.

Policy synchronization is decoupled through background workers, allowing threshold updates and retention/export changes without requiring a new app build.

## 3.3 Threat Model and Assumptions

The primary defensive goal is detection of suspicious encrypted traffic behavior such as beaconing, unusual destination changes, remote-control style communication, and malware-associated anomalies. The design assumes that payloads remain encrypted and unavailable for inspection.

The thesis considers three privacy-relevant adversaries:

- a **data consumer** who receives an exported or derived representation and attempts to infer application or behavior properties from that release;
- a **passive observer** who watches encrypted traffic on the network path and performs fingerprinting from sequence metadata alone;
- a **central coordinator** in collaborative training who may receive learned updates or reduced latent features rather than raw full-feature traffic.

The model does *not* assume root compromise of the endpoint OS, payload decryption on the wire, or strong traffic-shaping defenses such as dummy-packet padding.

Trust boundaries:

- **Trusted:** endpoint app process, local encrypted settings, backend adapter process.
- **Partially trusted:** network transport path (TLS-protected but exposed to outages).
- **Untrusted:** remote application peers and public network infrastructure.

## 3.4 Privacy Model

MANTA separates two privacy surfaces:

- **Release privacy:** what can be inferred from the feature representation that is intentionally exported, shared, stored, or used as the privacy-reduced model input.
- **Observer privacy:** what can still be inferred by a passive eavesdropper from the raw encrypted traffic sequence itself.

This distinction drives the system design. Feature hashing, identifier suppression, bucketization, privacy-student training, and federated latent sharing act on the released representation. A passive observer sees the original encrypted sequence before those transformations occur.

**Release privacy:** controls and audits the feature representation intentionally exported by MANTA. This surface is affected by hashing, removal of direct identifiers, bucketization, and privacy-student learning.

**Observer privacy:** concerns an on-path observer who sees encrypted-flow timing, volume, direction, and sequence patterns before MANTA applies export-time reductions.

**Boundary:** feature coarsening can reduce release leakage, but it cannot hide the original traffic pattern from an observer on the network path.

Figure 3.2: Difference between release privacy and observer privacy in MANTA.

## 3.5 Privacy-Preserving Design Decisions

The system applies data minimization and disclosure control at multiple points:

- metadata-only flow capture with no payload retention,
- hashed or removed direct identifiers in non-**off** privacy views,
- coarsened window features in the **medium** and **strict** release views,
- privacy-student training to learn a lower-leakage latent representation,

- federated-style training over privacy-student latents rather than raw full-feature tables.

These mechanisms provide measurable privacy-preserving properties, not formal guarantees. The evaluation measures how much leakage remains after each transformation.

## 3.6 Deployment and Evaluation Boundaries

The design remains anomaly-first rather than anonymity-first. MANTA does not implement traffic-shaping defenses such as padding, dummy traffic, or timing obfuscation. Its privacy mechanisms operate on the representation released by the endpoint or backend pipeline, while the observer benchmark measures what remains visible on the network path.

# Chapter 4

## Implementation

### 4.1 Android Endpoint Implementation

#### 4.1.1 VPN Capture and Userspace Forwarding

The endpoint capture loop is implemented around `FlowVpnService`, which establishes a TUN interface and starts a foreground service session. A dedicated userspace forwarder relays packets to upstream sockets while protecting those sockets from tunnel recursion via `VpnService.protect(...)` [5]. Parsed packets are aggregated into flow records keyed by app identity and network tuple attributes.

App attribution uses Android connection-owner resolution where available, and flows are persisted to Room-backed storage [4]. Background workers handle retention cleanup and export retries using `WorkManager` scheduling [6].

#### 4.1.2 Feature Windows and On-Device Detection

The feature window builder aggregates 60-second windows and computes a richer metadata-only feature set than the first prototype. In addition to core counters and ratios, the implemented pipeline derives timing, novelty, balance, diversity, burstiness, and graph-inspired contextual signals such as destination concentration, transition entropy, flow-count deviation, and byte-rate deviation. These features are used internally for anomaly scoring, but only a reduced subset is exposed in the privacy-reduced views.

On-device inference supports linear and TFLite-compatible paths. The Android-ready linear model is exported as a JSON artifact, while the one-class path exports a TFLite autoencoder [23]. These models are intentionally lightweight, but they serve mainly as feasibility baselines relative to the stronger remote models rather than as the final thesis headline.

Alert generation includes severity stabilization and false-positive budget enforcement, then

stores triage-ready records with explanation text and feature contributors.

### 4.1.3 UI and Triage Workflow

The Compose UI exposes:

- consent/disclosure and capture controls,
- backend URL/token and export toggle,
- alert center with triage states (OPEN, INVESTIGATING, RESOLVED, FALSE\_POSITIVE),
- local data purge and anonymized CSV snapshot export.

This supports both standalone local operation and analyst-assisted workflows.

## 4.2 Backend Adapter Implementation

The backend adapter is a FastAPI service with token authentication, strict request validation, and defense-in-depth response headers. Incoming events are enqueued in SQLite and forwarded with bounded retry/backoff; exhausted items move to a dead-letter queue for replay.

Beyond ingest, the adapter implements:

- alert triage APIs and incident summaries,
- per-device policy storage and auto-tuning hooks,
- policy simulation and quality summary endpoints,
- retraining sample export and forensics bundle export,
- optional Wazuh/Resistine connector routes [24].

This architecture isolates endpoint concerns from SIEM-specific integration complexity.

## 4.3 ML Pipeline Implementation

The ML pipeline is organized as reproducible command-line stages that now operate over a merged public corpus rather than only over a controlled synthetic trace:

- public-dataset normalization and merged-corpus manifest generation,

- grouped split audits and evaluation-protocol reports,
- on-device linear and TFLite export,
- remote backend training across multiple model families,
- privacy ablation and metadata leakage attack benchmarking,
- encrypted-flow sequence fingerprint benchmarking,
- privacy-student training and federated simulation,
- comparison-matrix and performance-gate generation.

### 4.3.1 Privacy Views

The privacy mechanism is implemented as explicit feature views rather than as an afterthought. `off` and `low` retain the full feature set, while `medium` and `strict` replace many exact values with coarse or distribution-aware buckets. For example, activity, volume, duration, rate, burstiness, stability, and shape are represented through a small number of bucketized values, while direct host hints and several stronger destination/timing cues are removed from the exported representation.

This is not packet padding or transport obfuscation. The system does not add dummy traffic. The privacy transformation happens at the feature-representation level.

### 4.3.2 Privacy Student and Federated Path

The privacy-student model is trained on the reduced `medium` view with reconstruction-style pretraining and multiple adversary heads. Those adversaries try to recover app identity, app family, dataset source, context bucket, and destination behavior from the learned latent representation. The objective is to preserve anomaly utility while reducing leakage of sensitive traffic attributes.

The federated simulator then trains over the privacy-student latent representation using a FedProx style algorithm rather than over raw full-feature windows. This does not prove secure deployment by itself, but it makes the collaborative-learning design concrete and measurable.

### 4.3.3 Reproducibility Support

The `run_experiment_suite` path, cached privacy-view derivation, manifest reports, and full comparison matrix are all designed to support reproducibility. The archived report set used in this manuscript was produced from the March 20–21, 2026 public-corpus run and is archived in the evidence bundle described in the appendix.

# Chapter 5

## Experimental Methodology and Evaluation

### 5.1 Experimental Setup

Primary quantitative results in this chapter are taken from the March 20–21, 2026 public-corpus report bundle generated under `ml-pipeline/reports/`. The submission snapshot mirrors the same report files under `exports/thesis-final-2026-03-22/reports/`. The evidence package includes dataset and protocol manifests, model reports, privacy ablation, metadata leakage, encrypted-flow fingerprinting, privacy-student and federated reports, performance gates, and the full comparison matrix.

The core principle of the evaluation is to avoid optimistic random-row validation. Model and privacy reports instead use grouped splits keyed by dataset source, environment, session, app family, and time bucket wherever the report type supports them. The separate `evaluation-protocol.json` audit covers six grouped split strategies over a sampled 250,000-window subset to verify that the methodology itself is source-aware.

### 5.2 Datasets and Trace Collection

The corpus is a public-only merge of seven normalized datasets: Android Mischief, Android Spyware, CICAndMal2017, ITC-Net-Blend-60 scenario E, a Parrot-derived metadata slice, SDNCampus, and Westermo. The merged corpus contains 6,034,487 flows and yields 904,371 60-second windows for feature extraction. [Table 5.1](#) summarizes the dataset composition reported by `dataset-manifest.json`.

The corpus spans malware-heavy Android traffic, benign campus/application traffic, and industrial network traces rather than a single-device lab trace. The resulting benchmark is harder than a controlled trace, but closer to the cross-source generalization problem

Table 5.1: Public-only corpus composition.

| Dataset source        | Flow rows | Sessions | Label profile          |
|-----------------------|-----------|----------|------------------------|
| Android Mischief      | 3,011     | 8        | anomaly-only slice     |
| Android Spyware       | 4,924     | 1        | mixed benign/anomalous |
| CICAndMal2017         | 2,587,130 | 2,119    | mixed benign/anomalous |
| ITC-Net-Blend-60 E    | 17,199    | 211      | benign-only            |
| Parrot metadata slice | 373,566   | 1        | benign-only            |
| SDNCampus             | 3,000,000 | 30       | benign-only            |
| Westermo              | 48,657    | 3        | mixed benign/anomalous |

MANTA is intended to study.

### 5.3 Metrics

- **Detection metrics:** precision, recall, F1, ROC-AUC, PR-AUC, and confusion matrix.
- **Privacy/utility metrics:** anomaly utility under `off`, `low`, `medium`, and `strict` views.
- **Metadata leakage metrics:** strongest-attacker accuracy and normalized leakage for app ID, app family, dataset source, context bucket, and destination behavior.
- **Observer leakage metrics:** encrypted-flow sequence fingerprinting accuracy and normalized leakage.
- **Performance metrics:** single-window and batch feature-extraction latency.

Normalized leakage is reported on a 0–1 scale after accounting for the baseline difficulty of the inference task. A value close to 0 means that the attacker is near the configured baseline or chance level for that task, while a value close to 1 means that the representation is highly identifying for the target attribute. The metric is therefore used as an empirical privacy-risk score rather than as a formal privacy guarantee.

### 5.4 Anomaly-Detection Results

Table 5.2 summarizes the main anomaly-detection variants from the full comparison matrix. The best overall detection result in the March 2026 matrix is the remote gradient-boosted-tree family benchmark, with F1 0.449, PR-AUC 0.482, and ROC-AUC 0.765. The on-device models are materially weaker, and none of the evaluated models clear the project’s high deployment gates.

The results show usable signal in metadata-only windows, especially for the remote tree model, but the measured operating points remain below the project gates. The

Table 5.2: Representative model results from the March 2026 report set.

| Model                                | Precision | Recall | F1    | PR-AUC / ROC-AUC |
|--------------------------------------|-----------|--------|-------|------------------|
| Android linear                       | 0.201     | 0.415  | 0.271 | 0.199 / 0.659    |
| TFLite one-class                     | 0.304     | 0.070  | 0.114 | 0.180 / 0.612    |
| Remote hybrid primary                | 0.352     | 0.271  | 0.306 | 0.375 / 0.690    |
| Remote family: logistic regression   | 0.455     | 0.292  | 0.356 | 0.301 / 0.691    |
| Remote family: gradient-boosted tree | 0.620     | 0.352  | 0.449 | 0.482 / 0.765    |
| Privacy student (medium)             | 0.156     | 0.383  | 0.222 | 0.139 / 0.589    |
| Federated (medium)                   | 0.114     | 0.919  | 0.203 | 0.125 / 0.548    |

privacy-oriented variants also lose substantial detection quality relative to the best remote benchmark.

## 5.5 Release-Privacy Utility Trade-off

The release-privacy ablation results are shown in Table 5.3. Contrary to the initial expectation that privacy-reduced views would necessarily hurt detection, the **medium** and **strict** views slightly *improve* F1 on this benchmark while keeping ROC-AUC and PR-AUC close to the **off** view. This suggests that reduced views can remove some source-specific noise or over-specificity while still preserving anomaly-relevant structure.

Table 5.3: Anomaly utility under privacy-reduced views.

| View          | Precision | Recall | F1    | PR-AUC / ROC-AUC |
|---------------|-----------|--------|-------|------------------|
| <b>off</b>    | 0.413     | 0.899  | 0.565 | 0.506 / 0.624    |
| <b>low</b>    | 0.413     | 0.899  | 0.565 | 0.506 / 0.624    |
| <b>medium</b> | 0.425     | 0.923  | 0.581 | 0.531 / 0.652    |
| <b>strict</b> | 0.425     | 0.930  | 0.582 | 0.495 / 0.631    |

## 5.6 Metadata Leakage from Released Views

Table 5.4 summarizes the strongest metadata leakage tasks for the main release-privacy views. The important result is mixed. Exact app-ID leakage drops substantially from the **off** view to **medium** and **strict**. However, app-family, destination-behavior, and dataset-source leakage remain high even in the reduced views. This is why the release-privacy gate for **medium** and **strict** still fails.

In both **medium** and **strict**, exact app re-identification falls below the configured threshold, but the app-family and destination-behavior checks remain false. The reduced views

Table 5.4: Strongest metadata leakage by privacy view (normalized leakage).

| View   | App ID | App family | Dataset source | Destination behavior |
|--------|--------|------------|----------------|----------------------|
| off    | 0.235  | 0.689      | 0.935          | 0.989                |
| low    | 0.233  | 0.659      | 0.825          | 0.976                |
| medium | 0.113  | 0.611      | 0.702          | 0.688                |
| strict | 0.099  | 0.616      | 0.702          | 0.685                |

therefore improve release privacy for one target while leaving broader semantic leakage unresolved.

## 5.7 Observer Leakage from Encrypted Traffic

The observer benchmark directly evaluates what a passive on-path observer could infer from the encrypted traffic sequence. This benchmark is deliberately separate from the release views, because the observer sees the original traffic before any export-time coarsening. Table 5.5 shows that observer leakage remains severe.

Table 5.5: Observer leakage from encrypted-flow sequence fingerprinting.

| Task           | Accuracy | Normalized leakage | Top-5 accuracy | Strongest model |
|----------------|----------|--------------------|----------------|-----------------|
| App ID         | 0.544    | 0.544              | 0.719          | MLP             |
| App family     | 0.810    | 0.783              | 0.979          | MLP             |
| Dataset source | 0.984    | 0.981              | 1.000          | MLP             |
| Context bucket | 0.046    | 0.044              | 0.107          | MLP             |

The reduced privacy views improve the representation that MANTA exports, but they do not protect against an observer who sees timing, burst structure, and sequence-level metadata on the network path. The experiment marks the boundary between feature-level privacy controls and transport-level traffic-analysis defenses.

## 5.8 Privacy Student, Federated Path, and Performance

The privacy-student model improves representation privacy only partially. In the archived **medium**-view report, anomaly quality remains low (F1 0.222, PR-AUC 0.139, ROC-AUC 0.589), while adversary accuracy is reduced strongly for app ID but remains notable for app family, dataset source, and destination behavior. The federated path is even weaker as a detector in the present configuration (F1 0.203, PR-AUC 0.125, ROC-AUC 0.548).

Performance gating is also incomplete. Table 5.6 summarizes the thesis-relevant gate outcomes. Single-window extraction meets the p95 target (22.5 ms) but misses the p99

target (87.9 ms versus the desired 40 ms). The feature-matrix construction itself is fast, but end-to-end single-window build time is not yet at the intended deployment threshold.

Table 5.6: Selected thesis gates and measured outcomes.

| Gate                            | Target                  | Measured outcome                               | Result |
|---------------------------------|-------------------------|------------------------------------------------|--------|
| Latency p95                     | $\leq 40$ ms            | 22.5 ms                                        | Pass   |
| Latency p99                     | $\leq 40$ ms            | 87.9 ms                                        | Fail   |
| Exact app-ID re-release leakage | below threshold         | <b>medium:</b> 0.113, <b>strict:</b> 0.099     | Pass   |
| Semantic release leakage        | below threshold         | app-family and destination leakage remain high | Fail   |
| Observer-leakage audit          | low fingerprint leakage | app ID 0.544, dataset source 0.981             | Fail   |
| Detector quality                | deployment-grade gates  | best F1 0.449                                  | Fail   |

## 5.9 Interpretation

The report set supports four findings and rejects one stronger interpretation.

- Metadata-only anomaly detection is technically feasible on a large public corpus.
- Remote models outperform the on-device, privacy-student, and federated variants in the March 2026 report matrix.
- Release-privacy transformations can reduce exact app re-identification while preserving reasonable utility.
- Observer leakage remains severe without transport-level defenses.
- MANTA should be treated as an evaluation framework and research artifact rather than as a product-ready privacy-preserving IDS.

# Chapter 6

## Privacy, Ethics, and Limitations

### 6.1 Privacy Impact

MANTA enforces a metadata-only telemetry boundary: packet payloads are not stored or exported, and the released views progressively suppress direct identifiers and strong destination hints. From a GDPR-oriented perspective [12], this aligns with data minimization, purpose limitation, and storage-limitation principles. The design still processes behaviorally revealing metadata, so transparency, consent, and retention control remain necessary rather than optional.

MANTA provides several privacy-preserving properties:

- metadata-only capture instead of payload inspection,
- hashed or removed direct identifiers in the reduced views,
- coarse or distribution-aware bucketization of many released features,
- privacy-student learning over a reduced feature view,
- federated-style learning over privacy-student latents instead of raw full-feature tables,
- explicit auditing of both release privacy and observer leakage.

These are concrete privacy-preserving *features* of the system, but they are not equivalent to a formal privacy guarantee.

### 6.2 Ethical Considerations

Responsible deployment requires:

- informed consent before monitoring starts,

- understandable disclosure of what is and is not collected,
- bounded retention with user-triggered purge capability,
- strict access controls on backend triage endpoints,
- constraints against repurposing telemetry for non-security surveillance.

These points are consistent with EDPB guidance on transparency and lawful processing in digital services [11]. Security hardening practices from mobile application standards (e.g., OWASP MASVS) further reduce secondary risk [20].

## 6.3 Release Privacy versus Observer Privacy

The release and observer measurements expose different failure modes. In the release view, **medium** and **strict** substantially reduce exact app-ID leakage. Those same views still leak high-level semantics such as app family and destination behavior. The observer benchmark then shows that a passive eavesdropper can recover strong information from encrypted-flow sequences even when the exported representation is reduced.

In concrete terms, a release-privacy defender controls the representation that is exported or shared. A passive observer does not consume that exported representation; they consume the original traffic behavior on the network path. This is why export-time coarsening does not neutralize traffic-analysis attacks. Works such as Deep Fingerprinting and subsequent evaluations of website fingerprinting defenses reinforce this distinction [15, 17].

## 6.4 Interpretation of the Negative Result

Feature-level coarsening is not enough to provide strong privacy against passive observers. Many metadata-only monitoring designs rely on payload omission as their primary privacy argument; the MANTA results show that this is incomplete. The released representation can be made less revealing, but encrypted traffic remains fingerprintable at the observer level.

## 6.5 Leakage and Utility Evaluation

MANTA benchmarks privacy with two complementary views:

- utility and leakage under feature-reduced release views, and
- observer leakage from encrypted-flow sequence fingerprinting.

The two measurements keep utility and inference risk visible at the same time: a reduced export format has to retain threat-detection signal while reducing what secondary attackers can infer.

## 6.6 Limitations and Threats to Validity

- The anomaly models do not yet reach deployment-grade quality under the configured gate thresholds.
- The dataset mix is public and heterogeneous but still inherits each source dataset’s own collection biases and labeling artifacts.
- The privacy-student and federated paths are measured prototypes, not production defenses.
- No packet padding, dummy traffic, or timing obfuscation is implemented, so passive-observer protection remains out of scope.
- Additional privacy attacks such as membership inference, poisoning, and adaptive counterfactual adversaries remain future work.

# Chapter 7

## Conclusion and Future Work

### 7.1 Conclusion

I presented MANTA, an end-to-end framework for metadata-only anomaly detection over encrypted mobile and endpoint traffic. The project combines Android capture, backend-assisted triage, privacy-aware feature views, privacy-student and federated variants, and a reproducible public-corpus evaluation pipeline. The results identify a practical boundary: metadata coarsening can improve release privacy, but it does not provide strong observer privacy against traffic-analysis attacks.

Answers to the research questions are:

- **RQ1 (metadata-only detection feasibility):** Metadata-only detection is feasible on a large public corpus, but the evaluated models remain below deployment-grade quality thresholds. The best detector in the March 2026 report matrix is a remote gradient-boosted-tree family variant with F1 0.449, PR-AUC 0.482, and ROC-AUC 0.765.
- **RQ2 (variant comparison):** Remote models outperform the evaluated on-device, privacy-student, and federated variants. The privacy-preserving variants are useful for measurement, but not yet competitive as detectors.
- **RQ3 (release privacy):** The reduced privacy views lower exact app-ID leakage while keeping anomaly utility broadly intact in the March 2026 ablation study. This supports a limited release-privacy result.
- **RQ4 (observer privacy):** The encrypted-flow sequence benchmark shows severe observer leakage. Release privacy and observer privacy are therefore not interchangeable, and feature-level coarsening alone is insufficient for strong traffic-analysis privacy.

I claim that MANTA is a reproducible encrypted-traffic IDS framework that separates release privacy from observer privacy and shows that metadata-only coarsening can reduce

exact app re-identification while remaining insufficient against stronger semantic and passive-observer inference.

Practically, the project delivers an artifact package: Android app, backend adapter, ML pipeline, report bundle, manuscript, and reproducibility appendix. Its value is strongest as an empirical systems study and evaluation framework rather than as a finished privacy-preserving product.

## 7.2 Future Work

- Stronger remote and on-device anomaly models under the same grouped public-corpus protocol.
- Transport-level defenses such as padding or timing obfuscation if observer privacy is a hard requirement.
- Better privacy-student objectives that suppress app-family and behavior leakage more strongly while preserving anomaly utility.
- More realistic deployment studies on mobile overhead, battery, and long-running user acceptance.
- Stronger collaborative-learning paths with secure aggregation, client weighting, and robustness against poisoning.

# Appendix A

## Reproducibility and Artifact Package

### A.1 Artifact Package

The main experiment artifacts used in this manuscript are the March 20–21, 2026 public-corpus report bundle. During the experiment run these reports were written to `ml-pipeline/reports/`; the submission snapshot mirrors the selected thesis reports under `exports/thesis-final-2026-03-22/reports/`:

- `ml-pipeline/reports/dataset-manifest.json`
- `ml-pipeline/reports/evaluation-protocol.json`
- `ml-pipeline/reports/android-model-evaluation.json`
- `ml-pipeline/reports/tflite-autoencoder-evaluation.json`
- `ml-pipeline/reports/remote-assisted-model.json`
- `ml-pipeline/reports/privacy-ablation.json`
- `ml-pipeline/reports/privacy-leakage.json`
- `ml-pipeline/reports/traffic-fingerprint.json`
- `ml-pipeline/reports/privacy-student-report.json`
- `ml-pipeline/reports/federated-report.json`
- `ml-pipeline/reports/performance-gates.json`
- `ml-pipeline/reports/privacy-gate.json`
- `ml-pipeline/reports/full-model-matrix.json`

## A.2 Core Reproduction Commands

The full public-corpus workflow is documented in `ml-pipeline/README.md`. The compact reproduction path is:

```
cd ml-pipeline
python -m pip install -e .[tfllite,test]

python -m ml_pipeline.dataset_manifest \
    --input data/real/manta-real-training.csv \
    --output reports/dataset-manifest.json

python -m ml_pipeline.evaluation_protocol_report \
    --input data/real/manta-real-training.csv \
    --output reports/evaluation-protocol.json

python -m ml_pipeline.train_all \
    --input data/real/manta-real-training.csv \
    --runs-root experiment-runs

python -m ml_pipeline.run_experiment_suite \
    --input data/real/manta-real-training.csv \
    --output-dir experiment-runs/real-run-001 \
    --model-threshold 0.5 \
    --ids-threshold 0.55
```

The normalized input CSV is derived from the public datasets listed in [Table 5.1](#). Raw datasets are not redistributed with the thesis archive; the archive contains the derived report bundle used for the manuscript tables.

# Bibliography

- [1] Mamoun Alazab et al. Enhancing privacy-preserving intrusion detection through federated learning. *Electronics*, 2023. URL <https://www.mdpi.com/2079-9292/12/16/3382>.
- [2] Tareq H. H. Aldhyani et al. Privacy-preserving fl-based ids for cyber-physical systems. *Mathematics*, 2024. URL <https://www.mdpi.com/2227-7390/12/20/3194>.
- [3] Android Developers. Foreground services overview. <https://developer.android.com/develop/background-work/services/foreground-services>, 2026. Accessed: 2026-02-28.
- [4] Android Developers. Save data in a local database using room. <https://developer.android.com/training/data-storage/room>, 2026. Accessed: 2026-02-28.
- [5] Android Developers. Vpnservice. <https://developer.android.com/reference/android/net/VpnService>, 2026. Accessed: 2026-02-28.
- [6] Android Developers. Workmanager. <https://developer.android.com/topic/libraries/architecture/workmanager>, 2026. Accessed: 2026-02-28.
- [7] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009. doi: 10.1145/1541880.1541882.
- [8] B. Claise. Cisco systems netflow services export version 9. Technical Report RFC 3954, IETF, 2004.
- [9] B. Claise, B. Trammell, and P. Aitken. Specification of the ip flow information export (ipfix) protocol for the exchange of flow information. Technical Report RFC 7011, IETF, 2013.
- [10] B. Claise, B. Trammell, and P. Aitken. Information model for ip flow information export (ipfix). Technical Report RFC 7012, IETF, 2013.
- [11] European Data Protection Board. Edpb guidelines 05/2020 on consent under regulation 2016/679. [https://edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-052020-consent-under-regulation-2016679\\_en](https://edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-052020-consent-under-regulation-2016679_en), 2020. Accessed: 2026-02-28.

- [12] European Union. Regulation (eu) 2016/679 (general data protection regulation). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016.
- [13] Xiaodan Gu and Kai Dong. Pd-pan: Prefix- and distribution-preserving internet of things traffic anonymization. *Electronics*, 12:4369, 2023. doi: 10.3390/electronics12204369.
- [14] IANA. Ip flow information export (ipfix) entities. <https://www.iana.org/assignments/ipfix/ipfix.xhtml>, 2026. Accessed: 2026-02-28.
- [15] Mohsen Imani, Marc Juarez, Payap Sirinam, and Matthew Wright. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018. doi: 10.1145/3243734.3243768.
- [16] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008. doi: 10.1109/ICDM.2008.17.
- [17] Nate Mathews, James K. Holland, Se Eun Oh, Mohammad Saidur Rahman, Nicholas Hopper, and Matthew Wright. Sok: A critical evaluation of efficient website fingerprinting defenses, 2022. Manuscript in the local thesis notes collection.
- [18] Yair Meidan, Michael Bohadana, Yisroel Mathov, Yisroel Mirsky, Daniel Breitenbacher, Asaf Shabtai, and Yuval Elovici. N-baiot: Network-based detection of iot botnet attacks using deep autoencoders. *IEEE Pervasive Computing*, 2018. URL <https://arxiv.org/abs/1805.03409>.
- [19] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. *arXiv preprint arXiv:1802.09089*, 2018. URL <https://arxiv.org/abs/1802.09089>.
- [20] OWASP Foundation. Owasp mobile application security verification standard (masvs). <https://mas.owasp.org/MASVS/>, 2025. Accessed: 2026-02-28.
- [21] Madushi H. Pathmapaperuma, Yogachandran Rahulamathavan, Safak Dogan, and Ahmet Kondo. Cnn for user activity detection using encrypted in-app mobile data. *Future Internet*, 14(2):67, 2022. doi: 10.3390/fi14020067.
- [22] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010. doi: 10.1109/SP.2010.25.
- [23] TensorFlow. Tensorflow lite guide. <https://www.tensorflow.org/lite/guide>, 2026. Accessed: 2026-02-28.

- [24] Wazuh. Wazuh api reference. <https://documentation.wazuh.com/current/user-manual/api/reference.html>, 2026. Accessed: 2026-02-28.
- [25] Yuxiang Wu et al. Et-ssl: Self-supervised learning for anomaly detection in encrypted traffic. *Scientific Reports*, 2025. URL <https://www.nature.com/articles/s41598-025-08568-0>.
- [26] Guoliang Xu, Ming Xu, Yunzhi Chen, and Jiaqi Zhao. A mobile application-classifying method based on a graph attention network from encrypted network traffic. *Electronics*, 12(10):2313, 2023. doi: 10.3390/electronics12102313.